ATLM

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA


# Advanced Techniques for Language Models

## Mini-Assignment 1

António Cruz (140129), Bruno Santos (140586), Pedro Miranda (129268)

# Index

# 1 Introduction

This report covers Mini-Assignment 1 of a three-stage project to build a domain-adapted language model for structured job posting generation. The full project motivation and pipeline are described in Section 2. The scope of this document is continued pretraining of SmolLM2-360M on 12,000 IT job descriptions, comparing full fine-tuning against LoRA as the controlled training knob. Sections cover data understanding, corpus construction, training setup, results, difficulties encountered, and conclusions.

# 2 Business Understanding

## 2.1 Context and Motivation

Recruitment is among the most resource-intensive processes in any organisation. In the United States, the SHRM 2025 report places the average cost per hire at $5,475 for non-executive roles and $35,879 for executive positions [9]; in the United Kingdom, the CIPD estimates £6,125 per hire [1]. The EURES portal alone publishes approximately 60 million vacancies per year across Europe [4], with each unfilled position costing an estimated $500 per day in lost productivity [9]. Shortages are most acute in healthcare, engineering, and IT [4]; precisely the sectors where job descriptions demand the most precision.

The challenge is compounded by a tightening regulatory environment. The EU Pay Transparency Directive (Directive (EU) 2023/970) requires all employers across the 27 member states to include salary ranges, use gender-neutral language, and drop salary-history questions by 7 June 2026 [8]. On the demand side, 52% of job seekers say description quality is very or extremely influential in their decision to apply [5], inclusive language yields up to 42% more responses [2], and 61% of candidates identify the salary range as the single most important element [7]. These pressures make structured, template-aware job description generation operationally necessary.

## 2.2 Work Plan

As described bellow, two publicly available datasets underpin all three stages: the **Djinni Recruitment Dataset** [3] (≈142,000 English IT job postings, 2020–2023) and the **LinkedIn Job Postings Dataset** [6] (≈124,000 cross-industry postings, 2023–2024). Djinni is used for training; LinkedIn is held out as an out-of-domain evaluation set throughout.

**Mini-Assignment 1 - Domain Adaptation.** A pretrained SmolLM2-360M model is continued pretrained on raw Djinni job descriptions using HuggingFace `Transformers`. The controlled experiment compares full fine-tuning against LoRA, measuring success via perplexity on held-out Djinni postings (in-domain) and LinkedIn postings (out-of-domain). The best checkpoint is carried forward to Mini-Assignment 2.

**Mini-Assignment 2 - Alignment.** The domain-adapted model cannot yet follow recruiter instructions. This stage adds supervised fine-tuning (SFT) on instruction–response pairs and Direct Preference Optimization (DPO), teaching the model to generate postings on command and prefer outputs that respect stated constraints, use inclusive language, and maintain structural completeness. Evaluation uses a three-way comparison (base, domain-adapted, aligned) on automatic metrics and qualitative analysis, implemented with HuggingFace `TRL`.

**Final Project - System Integration.** Two components are integrated on top of the aligned model: Retrieval-Augmented Generation (RAG) over the posting corpora, grounding outputs in real market conventions and salary ranges; and an LLM-as-Judge refinement loop that checks each draft against a structured rubric and regenerates below-threshold outputs. The system is evaluated against the Assignment 2 model and the unmodified base.

# 3 Data Understanding

This section summarises the exploratory analysis of both datasets. All exploratory plots (including keyword distributions, job family breakdowns, length histograms, and vocabulary comparisons) are available in the accompanying notebook. Only the figures most directly relevant to training decisions are reproduced here.

## 3.1 Dataset Overview

The primary corpus is the **Djinni Recruitment Dataset** [3]: 141,897 English-language IT job postings from the Djinni platform (2020–2023). Every description is present and unique (0% duplication), making it an unusually clean language-generation resource; the only notable missingness is the *English Level* field at 5.3%. The secondary corpus is the **LinkedIn Job Postings Dataset** [6]: 123,849 cross-industry postings from LinkedIn (2023–2024), held out entirely from training and used only for out-of-domain evaluation. Table 1 compares the two datasets on their target text fields.

| Dataset | Non-null | Unique | Dup. % | Mean len. | Median len. | Max len. |
|---|---|---|---|---|---|---|
| Djinni | 141,897 | 141,897 | 0.0% | 1,801 | 1,629 | 12,578 |
| LinkedIn | 123,842 | 107,827 | 12.9% | 3,766 | 3,435 | 23,201 |

Table 1: Target text quality comparison (character counts).

LinkedIn descriptions are roughly twice as long as Djinni's (3,766 vs. 1,801 characters mean) and carry a 12.9% duplicate rate versus 0% in Djinni. Both distributions are right-skewed with long tails beyond 5,000 characters, justifying a maximum token cap during tokenisation.

### 3.2 Domain, Role, and Structured Feature Analysis

Djinni spans 45 primary tech-domain keywords. The distribution is heavily skewed: JavaScript (17,903 postings), Java (8,712), DevOps (7,979), .NET (7,826), and Python (5,735) dominate, while niche domains such as Scala, Rust, and Blockchain have fewer than 500 entries. LinkedIn, by contrast, is genuinely cross-industry: 58% of its postings fall into an "Other" category outside IT-oriented keyword families, making it a meaningfully harder out-of-domain test set.

Djinni's structured metadata provides useful conditioning signals for later fine-tuning stages. Experience level (*Exp Years*) and English level together cover over 95% of rows and co-vary meaningfully: senior roles (5+ years) skew toward upper-intermediate and fluent English. Description length correlates with seniority (1,445 characters at entry level vs. 2,076 at 5+ years), indicating the model should learn to calibrate output length to the experience prompt. A regex-based skill extraction with 19 technical terms found that 81.8% of Djinni postings contain at least one recognised skill mention, with skills mapping consistently to expected domains (AWS - DevOps, Figma - Design, Kubernetes - DevOps) confirming learnable skill-to-role associations.

### 3.3 Tokenisation and Semantic Structure

Using the SmolLM2-360M tokenizer on a 5,000-posting sample, Djinni descriptions average 406 tokens (P95 = 796) and LinkedIn descriptions average 730 tokens (P95 = 1,495). A 1,024-token cap covers over 95% of Djinni postings without truncation and is used throughout training. Technical terms fragment heavily under the general-purpose tokenizer (e.g. "RabbitMQ" - 4 tokens, "Kubernetes" - 3 tokens), which motivates monitoring token budgets when setting sequence length caps.

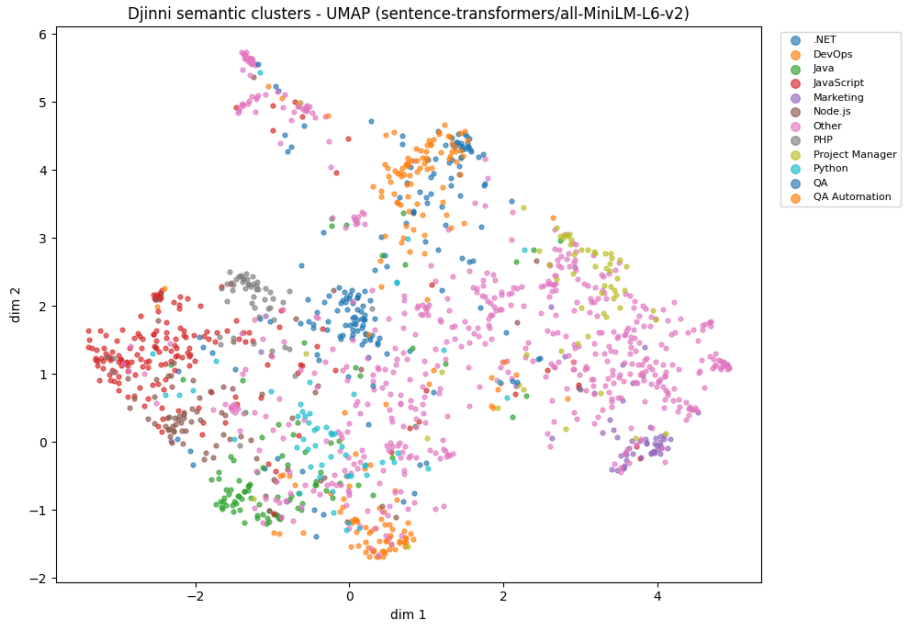


Figure 1: UMAP projection of 1,500 Djinni postings (all-MiniLM-L6-v2). Distinct clusters for .NEt, Java, JavaScript and Marketing confirm the dataset encodes learnable role structure.

A semantic embedding analysis of 1,500 Djinni postings (all-MiniLM-L6-v2 projected via UMAP) confirms that job descriptions form separable clusters by domain family, as shown in Figure 1. Nearest-neighbour inspection validates the structure: for a DevOps query, the five closest neighbours are all DevOps or Cloud-related (similarity 0.76-0.80), confirming the dataset supports domain-conditioned generation.

### 3.4 Summary and Implications for Training

Table 2 consolidates the key findings and their direct implications for the training pipeline.

| # | **Finding** | **Implication** |
|---|---|---|
| 1 | Djinni is IT-only; LinkedIn spans all industries | Use Djinni for train/val; LinkedIn as out-of-domain test only |
| 2 | Right-skewed description lengths (long tail >5k chars) | Apply max-token cap (∼1,024) during tokenisation |
| 3 | `Exp Years` and `English Level` cover >95% and co-vary | Include both as prompt fields for later fine-tuning stages |
| 4 | Description length correlates with seniority | Model should calibrate output length to experience level |
| 5 | Skills map consistently to expected roles | Dataset supports the mapping of skills to the expected roles |
| 6 | Semantic clusters are separable | Dataset structure supports domain-conditioned generation |

Table 2: Key data understanding findings and their implications for the training pipeline.

## 4 Data Preparation

This phase converts the raw datasets examined in Section 3 into the fixed training corpus that the continued-pretraining runs consume. The decisions below follow directly from the findings of Section 3.

### 4.1 Corpus Design

Continued pretraining requires only raw text, with no labels or instructions. Each record is stored as a single JSON line of the form `{"text": ...}`. The corpus is split into four disjoint files: training, validation, in-domain test, and out-of-domain test. All splits are drawn with random seed 42, making the corpus fully reproducible.

Training, validation, and in-domain test all draw from Djinni [3], ensuring that the model is measured against text from the same distribution it was trained on. The out-of-domain test uses deduplicated LinkedIn postings [6], which were never used for training; comparing in-domain

| Split | Documents | Size (MB) | Tokens (approx.) |
|---|---|---|---|
| Training | 12,000 | 22.7 | 5,387,796 |
| Validation | 1,000 | 1.9 | 452,643 |
| In-domain test | 1,000 | 1.9 | 447,152 |
| OOD test (LinkedIn) | 1,000 | 3.5 | 870,547 |

Table 3: Resulting corpus splits. Token counts are from the SmolLM2-360M tokenizer.

and out-of-domain perplexity answers the question of whether the adaptation is specific to IT or transfers more broadly.

### 4.2 Preprocessing

Three cleaning steps were applied before any split was drawn:

1. **Length filtering (Djinni).** Descriptions shorter than 200 characters (stub postings) and longer than 8,000 characters (outliers) were removed. This trims the right tail identified in Section 3 without discarding typical postings.

2. **Deduplication (LinkedIn).** The 12.9% of LinkedIn descriptions that were exact duplicates were dropped before the OOD test set was sampled, so the test set contains only distinct postings.

3. **Block packing.** After cleaning, descriptions are tokenized and concatenated with an EOS token between documents. The token stream is then sliced into fixed-length blocks of 1,024 tokens. This packing strategy avoids wasting context on padding and is the standard approach for continued pretraining on a language-model objective.

## 5 Modeling

### 5.1 Model Choice

The base model is **SmolLM2-360M**, a 360-million-parameter open-source GPT-style decoder released by HuggingFace. The choice was driven by three constraints. First, the model must fit and train on a single consumer GPU. We successfully tested training on NVIDIA RTX 5060 Ti(16 GB VRAM) and NVIDIA RTX 4090 (24 GB VRAM). Both cards running under WSL2 (using Ubuntu 24.04) on Windows 11; 360M parameters comfortably satisfies this constraint even with a full fine-tuning run. Second, the model must already produce grammatically coherent English text before any domain adaptation, since the goal is to shift the distribution rather than teach the model language from scratch; SmolLM2-360M meets this bar as a pretrained decoder. Third, the model must be large enough to produce readable, structured output; the smaller SmolLM2-135M variant was also tested as an ablation (Section 6) and produced noticeably lower-quality generations.

### 5.2 Experiment Design

The training optimization knob chosen for this assignment is **full fine-tuning versus LoRA** (Low-Rank Adaptation). Both approaches continue the model's pretraining on the same corpus using the same next-token-prediction objective; they differ only in how many parameters are updated. Full fine-tuning updates every weight in the model. LoRA freezes the original weights and inserts small low-rank adapter matrices next to the attention and feed-forward projections; only those adapter parameters are trained.

This comparison answers a practical question: can LoRA match or beat full fine-tuning on a domain-adaptation task while training only a small fraction of the parameters and taking less wall-clock time? Because this is the one varying factor, every other configuration (data, epochs, effective batch size, schedule shape) is held constant across both runs, making the comparison a controlled experiment.

### 5.3 Shared Hyperparameters

Table 4 lists the hyperparameters that are identical across both runs. The effective batch size of 16 was achieved by accumulating gradients over 4 micro-steps of size 4, a necessary compromise after LoRA raised a CUDA out-of-memory error at the initially intended micro-batch of 16 (discussed further in Section 7). Both runs use bfloat16 mixed precision, a linear warmup over the first 3% of steps, and cosine learning-rate decay. Training completes in 3 epochs over the 12,000-document corpus.

| **Parameter** | **Value** |
|---|---|
| Model | HuggingFaceTB/SmolLM2-360M |
| Block size | 1,024 tokens |
| Epochs | 3 |
| Per-device batch | 4 |
| Gradient accumulation steps | 4 (effective batch = 16) |
| Warmup ratio | 0.03 |
| Weight decay | 0.01 |
| Precision | bf16 |
| Random seed | 42 |
| Framework | HuggingFace `Transformers` + `Trainer` |

Table 4: Hyperparameters shared by both training runs.

### 5.4 Run-Specific Configuration

Table 5 shows the parameters that differ between the two runs. The learning rates reflect the conventional defaults for each method: LoRA typically trains well at a higher rate because the adapter matrices start from random initialisation, while full fine-tuning requires a lower rate to avoid destabilising pretrained weights.

| Parameter | Full fine-tuning | LoRA |
|---|---|---|
| Learning rate | $5 \times 10^{-5}$ | $2 \times 10^{-4}$ |
| Trainable parameters | 361.8M (100%) | 8.7M (2.3%) |
| LoRA rank $r$ | — | 16 |
| LoRA $\alpha$ | — | 32 |
| LoRA dropout | — | 0.05 |
| Target modules | — | `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj` |
| Wall-clock time | 42.1 min | 13.0 min |

Table 5: Per-run hyperparameters. LoRA targets all attention projections and both feed-forward layers in each transformer block.

### 5.5 Hardware and Reproducibility

All training and evaluation were conducted on both **NVIDIA RTX 4090** and **NVIDIA RTX 5060 Ti** GPU running under WSL2 on Windows 11. A single random seed (42) is fixed for corpus construction, training initialization, and evaluation sampling. Exact library versions are pinned in `requirements.lock.txt`. One caveat applies: some CUDA operations are non-deterministic, so perplexity values may vary slightly across runs even with the seed fixed; the variation is small and well below the differences on which conclusions are drawn.

## 6 Evaluation

This section evaluates the two training runs described in Section 5. The evaluation has three parts: training dynamics, test-set perplexity, and a qualitative inspection of generated text.

### 6.1 Training Dynamics

Figure 2 plots the training and validation loss for both runs across the three epochs. Both curves decrease steadily and level off without diverging, indicating stable optimisation and no overfitting on the 12,000-document corpus. LoRA converges to a lower validation loss (2.41) than full fine-tuning (2.57), despite training only 2.3% of the parameters, and does so in 13 minutes versus 42 minutes.

### 6.2 Test-Set Perplexity

Table 6 reports perplexity on the two held-out test sets for all three model variants. Lower perplexity means the model found the text more predictable.

Two findings stand out. First, both training methods substantially reduce in-domain perplexity: $-20\%$ for full fine-tuning and $-30\%$ for LoRA, confirming that continued pretraining on job-posting text measurably adapts the model to the recruitment domain. Second, out-of-domain perplexity barely moves ($-2\%$ and $+3\%$ respectively), showing that the adaptation is almost entirely specific to IT postings and does not transfer to the cross-industry LinkedIn set.

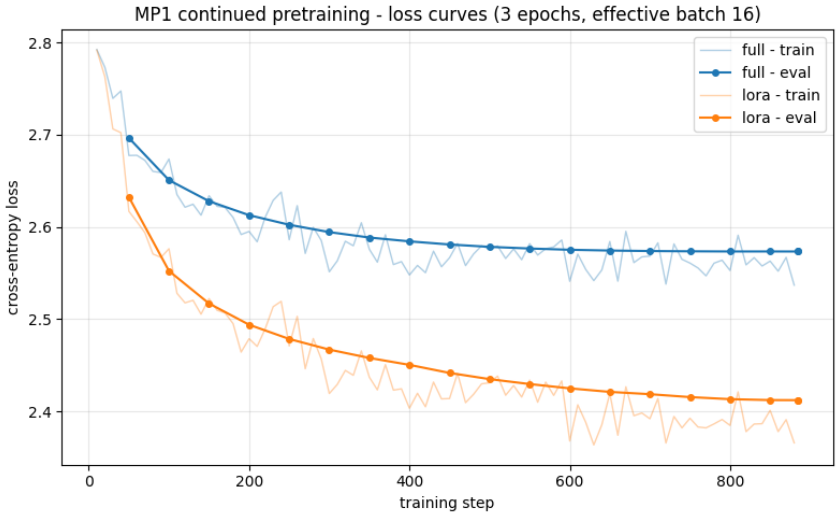


Figure 2: Training and validation loss for both runs (3 epochs, 12,000 training documents). Full fine-tuning in blue; LoRA in orange. Final losses. Full fine-tuning: training 2.537, validation 2.573. LoRA: training 2.366, validation 2.412.

| Model | In-domain ppl | vs. base | OOD ppl | vs. base |
|---|---|---|---|---|
| Base SmolLM2-360M | 16.37 | — | 17.80 | — |
| Full fine-tuning | 13.12 | -20% | 17.42 | -2% |
| LoRA $r$=16 | 11.38 | -30% | 18.27 | +3% |

Table 6: Test-set perplexity (lower is better). In-domain: 1,000 held-out Djinni postings. OOD: 1,000 LinkedIn postings, never seen during training.

This result is discussed further in Section 7.

One important caveat applies to the full versus LoRA comparison: the two runs differ not only in their optimisation method but also in their learning rate (5e-5 vs. 2e-4), each the conventional value for its method. Any gap in their perplexity therefore reflects the combined effect of method and learning rate and cannot be attributed to the method alone.

### 6.3 Model-Size Comparison: 135M vs. 360M

The experiment was also run with the smaller SmolLM2-135M checkpoint, using the same corpus and configuration. Table 7 places the two sizes side by side.

| | SmolLM2-135M | | SmolLM2-360M | |
|---|---|---|---|---|
| Variant | In-domain | OOD | In-domain | OOD |
| Base | 20.43 | 23.97 | 16.37 | 17.80 |
| Full fine-tuning | 16.24 | 23.09 | 13.12 | 17.42 |
| LoRA $r$=16 | 13.64 | 23.94 | 11.38 | 18.27 |

Table 7: Test-set perplexity for SmolLM2-135M and SmolLM2-360M (lower is better).

The 360M model achieves lower perplexity than the 135M model on every variant. The OOD gap is notably larger: the 135M base scores 23.97 versus 17.80 for the 360M base. Increasing

model size and domain adaptation are complementary interventions: the former improves general language modelling quality while the latter shifts the in-distribution vocabulary. The LoRA 360M checkpoint is therefore selected as the starting point for Mini-Assignment 2.

### 6.4 Qualitative Generation Analysis

To complement the perplexity numbers, Table 8 shows greedy-decoded completions from two prompts used uniformly across all three models. Greedy decoding (always choosing the most likely next token) makes the comparison consistent with the perplexity measurement, though sampled outputs read more naturally.

| **Prompt** | **Base** | **Full fine-tuning** | **LoRA** $r$=16 |
|---|---|---|---|
| *"We are looking for a"* | "...a way to make the most of the data we have." [×4 repetitions] | "...a Senior Software Engineer to join our team. We are looking for a Senior Software Engineer to join our team." [looped] | "...a Senior Full Stack Developer to join our team. **Requirements:** - 3+ yrs exp. - React.js - Node.js - AWS - Docker - Git - CI/CD" |
| *"The ideal candidate will"* | "...be a person willing to work hard ...work well in a team ...communicate effectively." (generic HR boilerplate) | "...be able to: • Understand software dev. process • Work independently • Work in fast-paced env." (vague, trailing bullet) | "...have a strong understanding of: - JavaScript/TypeScript - React - Redux - GraphQL - Docker - AWS - Git - CI/CD - GitLab" |

Table 8: Greedy-decoded completions (80 new tokens) for two fixed prompts, one per row.

The base model does not recognise either prompt as a job-posting fragment. Both adapted models immediately recognise the domain: full fine-tuning produces on-topic text but falls into repetition loops, while LoRA consistently produces structured, technology-specific output with correct formatting conventions. The qualitative picture aligns with the perplexity results.

## 7 Difficulties Encountered

**LoRA out-of-memory error.** LoRA raised a CUDA out-of-memory error at the initially intended micro-batch of 16. The resolution was to lower the micro-batch to 4 and compensate with gradient accumulation over 4 steps, keeping the effective batch size at 16. Both runs use this setting, so the fairness of the comparison is unaffected. What we would do differently: start from the lower micro-batch and increase only if memory permits, rather than discovering the limit at runtime.

**Attention-backend fallback on newer GPU (cross-machine memory discrepancy).** The same notebook used 10 GB on an RTX 4090 but peaked near 19 GB on an RTX 5060 Ti, because PyTorch's Windows wheels for the newer GPU lacked the FlashAttention kernel and fell back to a memory-heavy math implementation. Running the identical stack under WSL2, whose Linux wheels ship the kernel, restored the 10 GB footprint.

**Breaking library API change.** Version 5.9 of `transformers` removed the `overwrite_output_dir` argument from `TrainingArguments`, causing the first version of the training script to fail immediately. This is a reminder that fast-moving deep-learning libraries can break code between minor versions, which is why all dependency versions are pinned in `requirements.lock.txt`.

**Output-file collision across model sizes.** Running the experiment at two model sizes initially caused the second run to overwrite the results of the first, because output paths did not include a run-identity tag. The fix was a per-run output-folder scheme (`outputs/mp1-135m/` and `outputs/mp1-360m/`). What we would do differently: design the multi-run output layout from the outset, before any training begins.

**Near-zero out-of-domain transfer (unexpected finding).** Continued pretraining reduced in-domain perplexity by 20–30%, but out-of-domain perplexity on the cross-industry LinkedIn set moved by at most 3%. We had expected at least modest transfer, since all job postings share a common structural template. In practice the adaptation was almost entirely specific to IT vocabulary. This finding shows that continued pretraining on a domain-specific corpus tunes the model's distribution precisely to that domain and does not generalise to structurally similar neighbouring distributions

# 8 Conclusions

Continued pretraining on 12,000 IT job descriptions measurably adapts SmolLM2-360M to the recruitment domain. Both training methods achieve the goal: full fine-tuning reduces in-domain perplexity by 20% and LoRA by 30%, while out-of-domain perplexity barely moves, confirming the adaptation is IT-specific rather than structural. LoRA achieves the better result while training only 2.3% of the parameters in 13 minutes versus 42 for full fine-tuning, though the learning-rate difference means this gap cannot be attributed to the method alone. A secondary experiment with SmolLM2-135M shows that model scale and domain adaptation are complementary: the 360M model benefits less from adaptation in both absolute and relative terms. The LoRA 360M checkpoint is selected as the starting point for Mini-Assignment 2, where SFT and DPO will add instruction-following and constraint-respecting behaviour on top of the domain knowledge built here.

# Bibliography

[1] Chartered Institute of Personnel and Development. *Resourcing and Talent Planning Survey*. Average UK cost per hire: £6,125. As cited in StandOut CV, "Recruitment Statistics in the UK 2026". 2025. URL: https://standout-cv.com/stats/recruitment-statistics-uk.

[2] Compono. *Inclusive Language in Job Descriptions*. Job descriptions with gender-neutral language receive up to 42% more responses. As cited in AssessCandidates, "How to Write Engaging Job Descriptions That Stand Out". 2025. URL: https://www.assesscandidates.com/writing-engaging-job-descriptions/.

[3] Nazarii Drushchak and Mariana Romanyshyn. "Introducing the Djinni Recruitment Dataset: A Corpus of Anonymized CVs and Job Postings". In: *Proceedings of the Third Ukrainian Natural Language Processing Workshop (UNLP) @ LREC-COLING 2024*. Dataset available at https://huggingface.co/datasets/lang-uk/recruitment-dataset-job-descriptions-english. Approximately 142,000 English job descriptions from the Djinni IT job platform (2020–2023). Torino, Italia: ELRA and ICCL, May 2024, pp. 8–13. URL: https://aclanthology.org/2024.unlp-1.2.

[4] European Labour Authority. *EURES Report on Labour Shortages and Surpluses 2024*. Prepared by Cambridge Econometrics. 60 million vacancies published yearly via EURES. Severe shortages in healthcare, engineering, IT, and construction across 31 EURES countries. European Labour Authority, 2025. URL: https://www.ela.europa.eu/en/publications/labour-shortages-and-surpluses-europe-2024.

[5] Indeed. *Job Description Best Practices*. 52% of job seekers say JD quality is "very" or "extremely" influential. 63% declined to apply due to unclear skill requirements. As cited in Workwolf, "2024 Hiring Stats Every Employer Needs to Know". 2024. URL: https://workwolf.com/blog/2024-hiring-stats-every-employer-needs-to-know/.

[6] Arsh Konforti. *LinkedIn Job Postings Dataset*. Approximately 124,000 job postings with structured metadata including title, company, description, salary range, and location. 2024. URL: https://www.kaggle.com/datasets/arshkon/linkedin-job-postings.

[7] LinkedIn. *Talent Solutions Survey*. 61% of candidates say the salary range is the most important part of a job description. As cited in Applicantz, "How to Write Inclusive Job Descriptions That Attract Diverse Talent". 2025. URL: https://applicantz.io/how-to-write-inclusive-job-descriptions-that-attract-diverse-talent/.

[8] Ogletree Deakins. *The EU Pay Transparency Directive's Progress Explained*. Directive (EU) 2023/970. Transposition deadline: 7 June 2026. Employers must disclose salary ranges in postings, ban salary history questions, use gender-neutral job titles. Jan. 2026. URL: https://ogletree.com/insights-resources/blog-posts/the-eu-pay-transparency-directives-progress-explained/.

[9] Society for Human Resource Management. *2025 Talent Acquisition Benchmarking Report*. Average cost per hire: $5,475 (non-executive), $35,879 (executive). Vacancy cost: $500/day. SHRM, 2025. URL: https://www.shrm.org/.